



SAP Application Development (S/4 HANA)

Analytical App Development

Motivation

This chapter will show you how to develop an analytical app based on OData and using the corresponding SAP UI5 controls. Therefore, you will learn how to make your entities from OData service consumable for the smart controls.

Prerequisites

Before starting this exercise, you should have fundamental ABAP programming skills and be familiar with the Eclipse IDE. Furthermore, it is recommended that you have completed the *HTML5, CSS & JavaScript*, *SAPUI5*, and OData chapters. The completion of the chapter *SAP Fiori* is also helpful.

Content

This chapter explains the necessary steps to build an analytical app. First, an OData service that provides the data for a simple chart will be created and exposed. Second, a web application to consume this service will be built in SAP UI5. In the next step a service, capable of providing the data for a smart chart will be created and configured. The application will be extended with the smart chart. Finally, the application will be deployed to the SAP backend.

Lecture notes

Students can go on with their accounts from the previous chapters mentioned in the prerequisites. If you skipped these chapters, your students may need to enter a developer key when creating their services. Please consult the lecture notes for further assistance on this topic. Furthermore, students should always use an individual three-digit number to mark their repository objects. In this exercise, these are displayed as ### and have to be replaced by the students' individual numbers. In addition, please replace \$\$ with your two-digit institution id.

- **Product:** All
- **Level:** Advanced
- **Focus:** Programming
- **Version:** 2.0
- **Author:** UCC Technical University of Munich

Task 1: Implement the OData service for a simple chart

Short description: Using the integrated SAP GUI to create a new project in the Gateway Service Builder and generate the necessary runtime objects

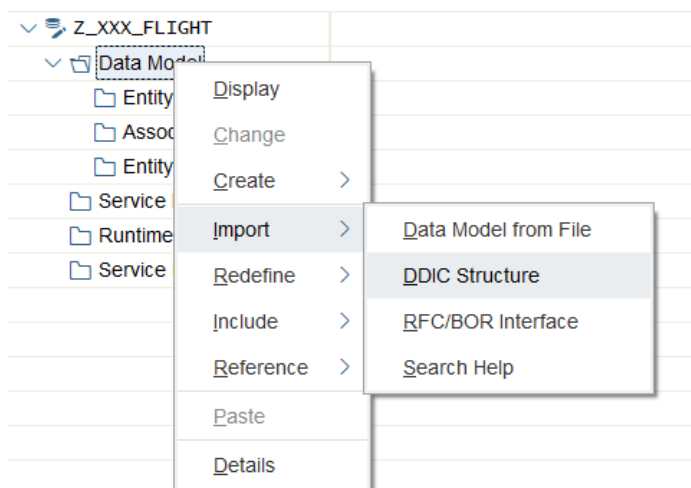
Start Eclipse, and open the ABAP perspective. Open the integrated SAP GUI in Eclipse by clicking on , selecting an existing ABAP project and clicking **OK**. If you currently have no existing ABAP project, please refer to the *Introduction to SAP Programming* chapter on how you create a new ABAP project.

In the integrated SAP GUI, call transaction code **SEGW**, which opens the Gateway Service Builder. For this task we will use your existing gateway project **Z_\$\$_###_FLIGHT** that you have created in the exercise on OData. When you call the transaction, it should

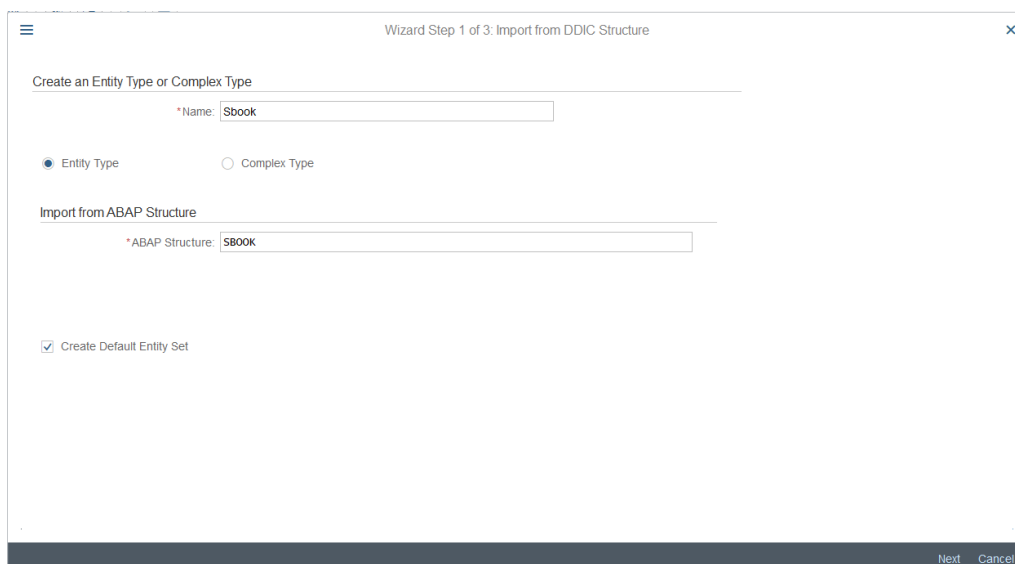
Open Gateway Service Builder

be already there. If not, click on Open Project , enter the project name, and click on Execute (in case you did not complete that exercise, create a project with this name. Refer to the aforementioned exercise if you experience any difficulties)

Switch to change mode by clicking on , otherwise you will not be able to create new entities. Expand your service and import the structure for the entity that we are going to use by right-clicking on **Data Model** and following the path **Import → DDIC Structure**.



Our entity is called Sbook based on Entity Type SBOOK. Make sure that the “Create Default Entity Set” checkbox is checked.



Since we are going to visualize the revenue per airline, select two fields – **Carrid**, and **Loccuram**, first containing the airline id, and the second one containing the cost of the single booking, stored in this table.

Data Source Parameter	Assign Structure	Description	Type	Length	Decimals	Import Search Help	Search Help	Output Length	Conv...
☒ CARRID	☐	Airline	CHAR	3		☐	H_SCARR	3	
☐ CONNID	☐	Flight Number	NUMC	4		☐	H_SPFLI	4	
☐ FLDATE	☐	Date	DATS	8		☐	SFLIGHT	10	
☐ BOOKID	☐	Booking number	NUMC	8		☐		8	
☐ CUSTOMID	☐	Customer Number	NUMC	8		☐		8	
☐ CUSTTYPE	☐	B/P customer	CHAR	1		☐		1	
☐ SMOKER	☐	Smoker	CHAR	1		☐		1	
☐ LUGGWEIGHT	☐	Luggage Weight	QUAN	8	4	☐		10	
☐ WUNIT	☐	Unit of measure	UNIT	3		☐	H_T006	3	CUNIT
☐ INVOICE	☐	Invoice pty.	CHAR	1		☐		1	
☐ CLASS	☐	Class	CHAR	1		☐		1	
☐ FORCURAM	☐	Amount	CURR	15	2	☐		20	
☐ FORCURKEY	☐	Payment currency	CUKY	5		☐		5	
☒ LOCCURAM	☐	Amount	CURR	15	2	☐		20	
☐ LOCCURKEY	☐	Airline Currency	CUKY	5		☐		5	
☐ ORDER_DATE	☐	Booking date	DATS	8		☐		10	

Mark Carrid as key and click on Finish.

IsEnt	Complex/Entity Type Name	ABAP Name	Is Key	Type	Name	Label
☒	Stbook	CARRID	☒	CHAR	Carrid	Airline
☒	Stbook	LOCCURAM	☐	CURR	Loccuram	Amount

Save your project by clicking on **Save** and generate the runtime artifacts . Choose your transport request and leave the proposed class names without changes. Choose your package and the same transport request when prompted again. Your service is ready for the implementation now. Expand the **runtime artifacts** node, right click on your **...DPC_EXT** class, and select **Go to ABAP Workbench**.

- ☐ Z_XXX_FLIGHT
 - ☐ Data Model
 - ☐ Service Implementation
 - ☒ Runtime Artifacts
 - ☐ Z_XXX_FLIGHT_MDL
 - ☐ Z_XXX_FLIGHT_SRV
 - ☐ ZCL_Z_XXX_FLIGHT_DPC
 - ☒ ZCL_Z_XXX_FLIGHT_DPC_EXT
 - ☐ ZCL_Z_XXX_FLIGHT_DPC_EXT
 - ☐ ZCL_Z_XXX_FLIGHT_DPC_EXT
 - ☐ ZCL_Z_XXX_FLIGHT_DPC_EXT
 - ☐ Service Maintenance

Since we are only going to read the data from the database and not make any changes, the only method we need to redefine is **SBOOKSET_GET_ENTITYSET**. Like all the CRUD methods, it has to be defined in the **PROTECTED** section. Add it after the previously defined methods, if you completed the OData exercise or at the beginning of the section otherwise.

```
PROTECTED SECTION.
METHODS flightset_create_entity REDEFINITION.
METHODS flightset_get_entityset REDEFINITION.
METHODS flightset_get_entity REDEFINITION.

METHODS scarrset_get_entity REDEFINITION.
METHODS scarrset_get_entityset REDEFINITION.

METHODS sbookset_get_entityset REDEFINITION.
```

Add the method implementation. In this case we will be using the new Open SQL syntax to be able to write functions like SUM() or AVG(). This syntax requires the ABAP variables, referenced in the SQL call, to be escaped with @.

```
METHOD sbookset_get_entityset.

SELECT CARRID, SUM( LOCCURAM ) AS LOCCURAM
  INTO CORRESPONDING FIELDS OF TABLE @et_entityset
  FROM sbook
  GROUP BY carrid
  ORDER BY carrid.

ENDMETHOD.
```

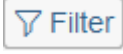
Save and activate your class.

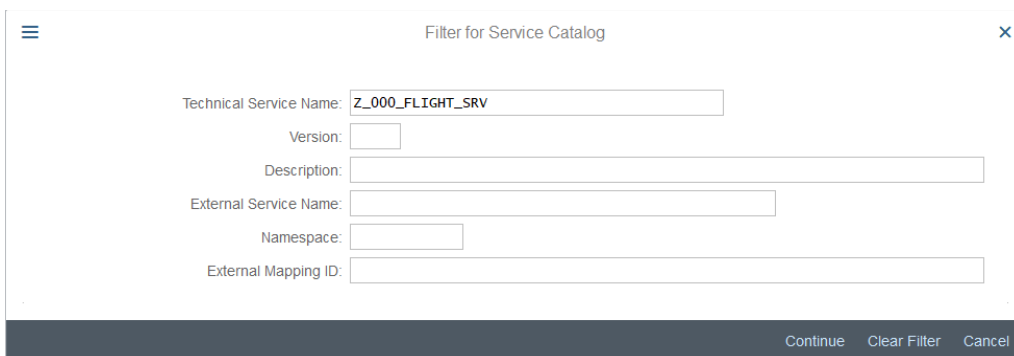
Task 2: Test the OData Service

Short description: Test your OData service by looking at its metadata and at the result the OData model

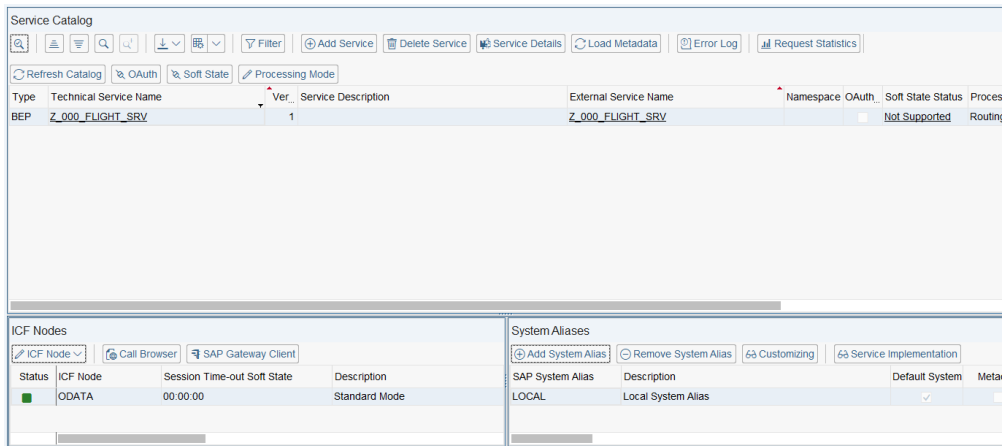
Since we are using an existing OData project and an existing service, we do not need to activate it again. If you did not complete the exercise on OData, it would be necessary to activate the **Z \$\$ ### FLIGHT_SRV** service. Refer to the Exercise 13 or the task 6 of this exercise for the instructions.

After the service has been activated, it will appear in the service catalog in the **/n/IWFND/MAINT_SERVICE** transaction. Since there are lots of services on an S/4

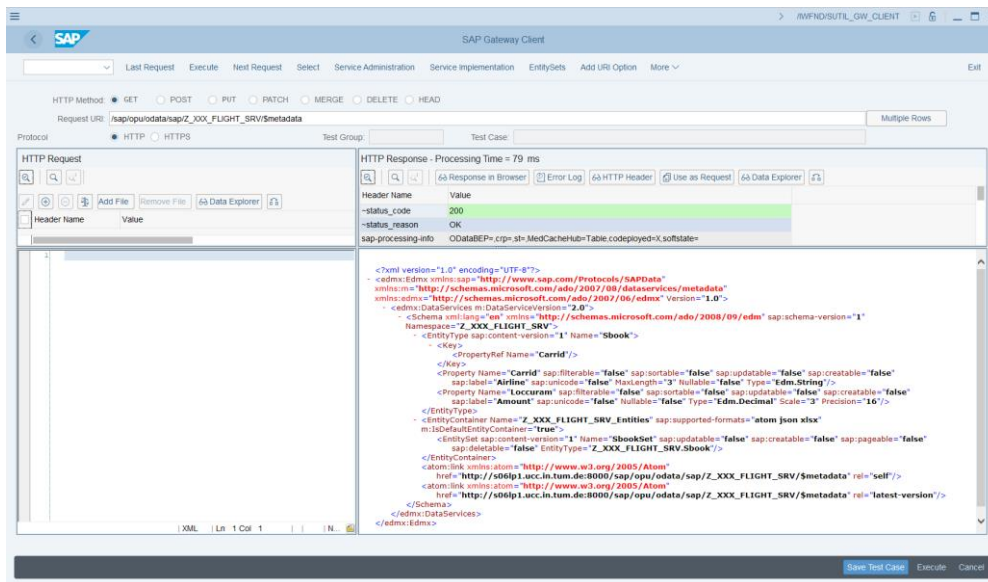
HANA system, click on Filter , enter the aforementioned service name, and click Continue.



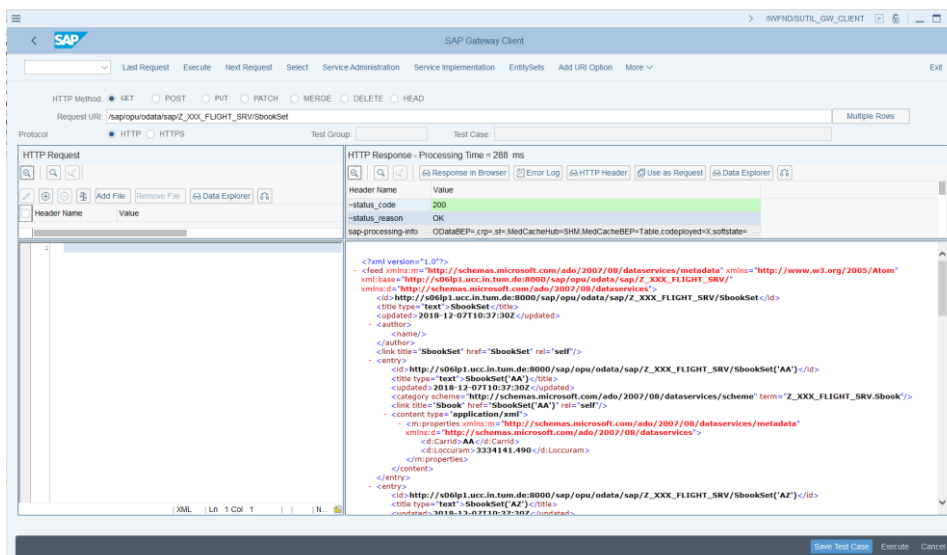
Your service will be the only one in the list.



Select it and click on SAP Gateway Client . There you can test the service. First, view the service metadata. Use Add URI Option button to adapt the request URI.



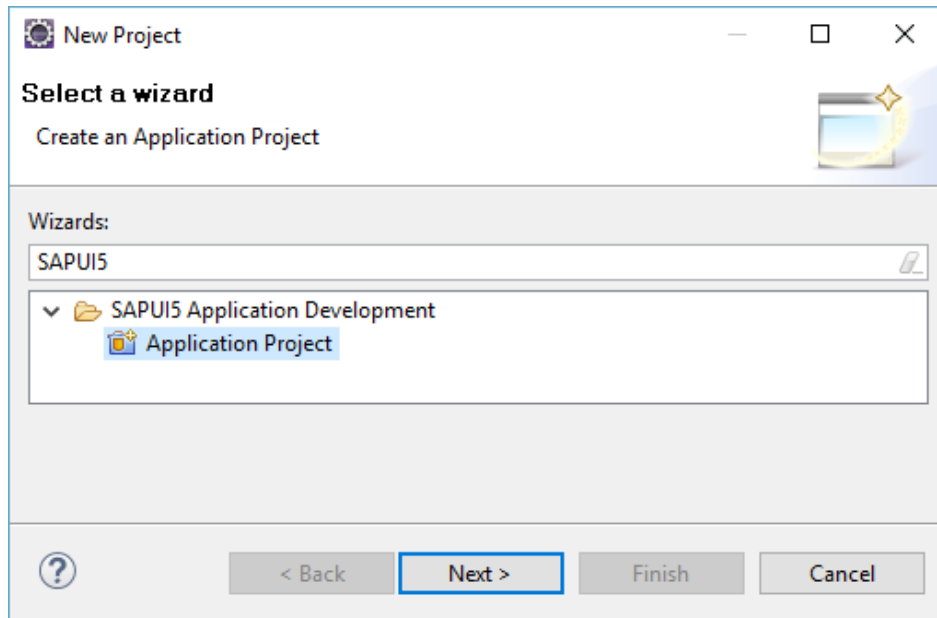
Now, display the contents of the SbookSet. Use Entity Sets button to add the set name to the request URI.



Task 3: Create a Fiori Component to consume the service

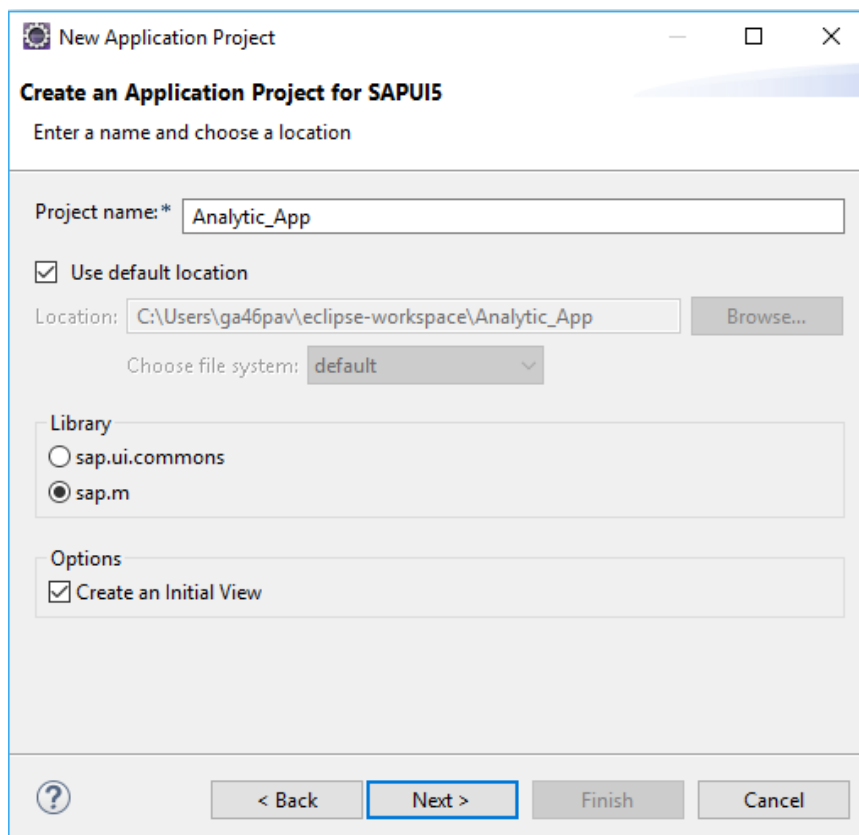
Short description: Creating a Fiori Component, containing a simple pie chart that visualizes the revenues of the airlines.

Back in Eclipse, we now want to create a SAPUI5 application which consumes our OData Services. In the context menu of the Project Explorer, choose **New->Other->SAPUI5 Application Development->Application Project**. Click **Next**.



Create SAPUI5 application

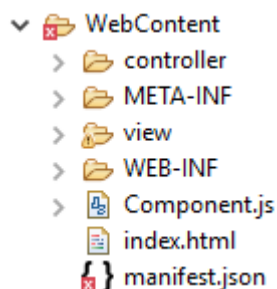
Give your project the name **Analytics_App** (no need to add your two-digit number at this point). Choose **sap.m** as library and check **Create an Initial View** and click **Next**.



On the next page, you have to define the properties for your initial view. Enter the name **main** and select to develop the view using **XML**.

Create a new view

After the project has been created, change your folder structure as depicted on a screenshot below – create the **view** and **controller** folders, as well as **Component.js** and **manifest.json** files (ignore the error in manifest.json for now). Move the main view and main controller into the corresponding folders. Delete the analytics_app folder.



Make the necessary changes to the index.html file (replace 000 with your number). Pay attention to the **src** property in the script definition. This time we are not using the UI5 libraries installed on the server, but load them from the SAP repository to make sure that we have a version, compatible with the controls we are using.

```
<!DOCTYPE HTML>
<html>
  <head>
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta http-equiv='Content-Type' content='text/html; charset=UTF-8' />

    <script
      src="https://sapui5.hana.ondemand.com/1.60.1/resources/sap-ui-core.js"
      id="sap-ui-bootstrap"
      data-sap-ui-theme="sap_bluecrystal"
      data-sap-ui-libs="sap.m"
      data-sap-ui-bindingSyntax="complex"
      data-sap-ui-compatVersion="edge"
      data-sap-ui-preload="async"
      data-sap-ui-resourceroots='{ "analytics_app_000": "." }'
    >
    </script>
    <!-- -->
    <script>

      sap.ui.getCore().attachInit(function () {
        sap.ui.require([
          "sap/m/Shell",
          "sap/ui/core/ComponentContainer"
        ], function (Shell, ComponentContainer) {
          new Shell({
            app : new ComponentContainer({
              name : "analytics_app_000",
              settings : {
                id : "webapp"
              }
            })
          }).placeAt("content");
        });
      });

    </script>

  </head>
  <body class="sapUiBody" role="application" id="content">
  </body>
</html>
```

Define your component in the **Component.js**. Again, replace 000 with your number.

```
jQuery.sap.declare("analytics_app_000.Component");
sap.ui.define([
  "sap/ui/core/UIComponent"
], function (UIComponent) {
  "use strict";

  return UIComponent.extend("analytics_app_000.Component", {

    metadata: {
      manifest: "json"
    },

    init: function () {
      UIComponent.prototype.init.apply(this, arguments);
      this.getRouter().initialize();
    },

    createContent: function(){
      var oViewData = {
        component : this
      };

      return sap.ui.view({
        viewName: "analytics_app_000.view.App",
        type: sap.ui.core.mvc.ViewType.XML,
        viewData: oViewData
      });
    }
  });
});
```

Implement the application metadata in the manifest.json file. Again, replace 000 with your number and use your service URL when defining a data source.


```
{
  "sap.app": {
    "_version": "1.1.0",
    "id": "analytics_app_000",
    "type": "application",
    "applicationVersion": {
      "version": "1.1.0"
    },
    "title": "Flight Monitor",
    "dataSources": {
      "dataForDashboard": {
        "uri": "/sap/opu/odata/sap/Z_XXX_FLIGHT_SRV/",
        "type": "OData",
        "settings": {
          "odataVersion": "2.0"
        }
      }
    }
  },
  "sap.ui5": {
    "rootView": {
      "viewName": "analytics_app_000.view.main",
      "type": "XML",
      "async": true
    },
    "models": {
      "dataForDashboard": {
        "dataSource": "dataForDashboard",
        "preload": true
      }
    },
    "routing": {
      "config": {
        "viewPath": "analytics_app_000.view",
        "controlAggregation": "pages",
        "routerClass": "sap.m.routing.Router",
        "viewType": "XML",
        "transition": "slide",
        "async": true
      },
      "routes": [
        {
          "pattern": "",
          "name": "home",
          "target": "home"
        }
      ],
      "targets": {
        "home": {
          "viewId": "main",
          "viewName": "main",
          "controlId": "app",
          "viewLevel": 1
        }
      }
    }
  }
}
```

The only missing file to make the project runnable is App view. Like in the OData exercise, no controller is needed.

```
<mvc:View
  xmlns="sap.m"
  xmlns:mvc="sap.ui.core.mvc"
  displayBlock="true">
  <App id="app"/>
</mvc:View>
```

Now open the main.view.xml. Here we will display our charts. At first, change the controller name, replacing 000 with your number:

```
controllerName="analytics_app_000.controller.main"
```

After that change the title of the application to **Dashboard** by editing the **title** property of the **Page** element. To display the chart, we will use the **FixFlex** container element. It is divided into two sections – fix content and flex content. Fix content will contain the control panel of the pie chart, and the chart itself with the corresponding chart container will belong to the flex content. Define the component in the Page content.

```
<layout:FixFlex id="chartContainer">
  <layout:fixContent>

  </layout:fixContent>
  <layout:flexContent>

  </layout:flexContent>
</layout:FixFlex>
```

Now, define the chart inside of a flex content.

```
<VBox fitContainer="true">
  <viz:Popover id="pieDetails"></viz:Popover>
  <viz:VizFrame id="revenuePerCarriidPie"
    uiConfig="{applicationSet:'fiori'}" height='100%' width="100%"
    vizProperties="{title: {text : 'Revenue per Airline', visible: true}}"
    vizType='pie'>
    <viz:dataset>
      <viz.data:FlattenedDataset
        data="{dataForDashboard}/SbookSet">
        <viz.data:dimensions>
          <viz.data:DimensionDefinition
            name="Airline" value="{Carriid}" />
        </viz.data:dimensions>
        <viz.data:measures>
          <viz.data:MeasureDefinition
            name="Total Revenue" value="{Loccuram}" />
        </viz.data:measures>
      </viz.data:FlattenedDataset>
    </viz:dataset>
    <viz:feeds>
      <viz.feeds:FeedItem uid="size" type="Measure"
        values="Total Revenue" />
      <viz.feeds:FeedItem uid="color"
        type="Dimension" values="Airline" />
    </viz:feeds>
  </viz:VizFrame>
</VBox>
```

Define the switch that allows to display the percentage on the chart in the fix content section.

```
<HBox class='settingsHBox'>
  <VBox class='settingsBox'>
    <Label text='Show Percentage' design="Bold"
      class='settingsLabel'></Label>
    <Switch id='percentageDisplay' class='settingsSwitch'
      state="false" change='onPercentageChanged'>
      <layoutData>
        <FlexItemData growFactor="1" />
      </layoutData>
    </Switch>
  </VBox>
</HBox>
```

Add the required dependencies to the header of the view definition.

```
xmlns:viz="sap.viz.ui5.controls" xmlns:layout="sap.ui.layout"
xmlns:viz.data="sap.viz.ui5.data"
xmlns:viz.feeds="sap.viz.ui5.controls.common.feeds">
```

The view is now ready, we just need to save it by clicking on one of the Save buttons



and to implement the application logic. Open the main controller file and define it as depicted below.

```
sap.ui.define([
    'sap/ui/core/mvc/Controller',
    'sap/ui/core/util/MockServer',
    'sap/suite/ui/commons/ChartContainer',
    'sap/suite/ui/commons/ChartContainerContent',
    'sap/viz/ui5/format/ChartFormatter',
    'sap/viz/ui5/api/env/Format'
], function (Controller, MockServer, ChartContainer, ChartContainerContent, ChartFormatter,
    Format) {
    "use strict";

    return Controller.extend("analytics_app.controller.main", {

        getRouter : function () {
            return sap.ui.core.UIComponent.getRouterFor(this);
        },

        onInit: function () {
            var that = this;

            this.setupContainer();
            this.configurePopover();
        }

    });
});
```

You might notice that a lot of dependencies have been imported into the controller. They will all be necessary later. For now, you have defined a function `getRouter()`, necessary for the in-app navigation, and called the two set-up functions in the `onInit` method. Now we are going to implement these functions.

As presented in the slides, the charts from the viz-library have no toolbars or any interaction possibilities active per default. To enable them we need to place the chart in a chart container. This is done programmatically – implement the function `setupContainer` after the `onInit`:

```
setupContainer: function() {
    var oVizFrame = this.getView().byId("revenuePerCarriidPie");
    var oChartContainerContent = new ChartContainerContent({
        icon : "sap-icon://pie-chart",
        title : "Revenue per Airline",
        content : [ oVizFrame ]
    });
    var oChartContainer = new ChartContainer({
        content : [ oChartContainerContent ]
    });
    oChartContainer.setShowFullScreen(true);
    oChartContainer.setAutoAdjustHeight(true);
    oChartContainer.setShowZoom(false);
    this.getView().byId('chartContainer').setFlexContent(oChartContainer);
},
```

Now setup the popover, that will be displayed when the user clicks on a value in a pie chart:

```
configurePopover: function(){
    Format.numericFormatter(ChartFormatter.getInstance());
    var oPopover = this.getView().byId("pieDetails");
    var oVizFrame = this.getView().byId("revenuePerCarriidPie");
    oPopover.connect(oVizFrame.getVizUid());
    oPopover.setFormatString(ChartFormatter.DefaultPattern.STANDARDFLOAT);
},
```

In the last step, program the event handler for the switch in a way that when it is switched on the percentage labels are displayed on the chart.

```
onPercentageChanged : function(oEvent){
    this.getView().byId("revenuePerCarriidPie").setVizProperties({
        plotArea: {
            dataLabel: {
                visible: oEvent.getParameter('state')
            }
        }
    });
},
```

Finally, save your JavaScript file by clicking on .

Task 4: Deploy your application to the SAP backend

Short description: Deploying the application to the SAP backend to make it accessible on the server

To deploy your application to the SAP backend, right-click on your web project in the project explorer and choose **Team->Share Project**. In the next step, select **SAPUI5 ABAP Repository** as repository type. Afterwards, choose your system connection, and enter User and Password if necessary. Next, define to create a new BSP Application. Enter Name **Z_\$\$_###_Analytics**, a description, and your Package to which you want to assign your application. Click **Next**.

**Deploy
application to
SAP backend**

Choose a transport request. Again, you may use your default transport request you used for other exercises. Click on **Finish**. The application will be deployed to the SAP backend. If you encounter any difficulties when deploying, consult the exercises on UI5 and OData.

**Submit
repository objects**

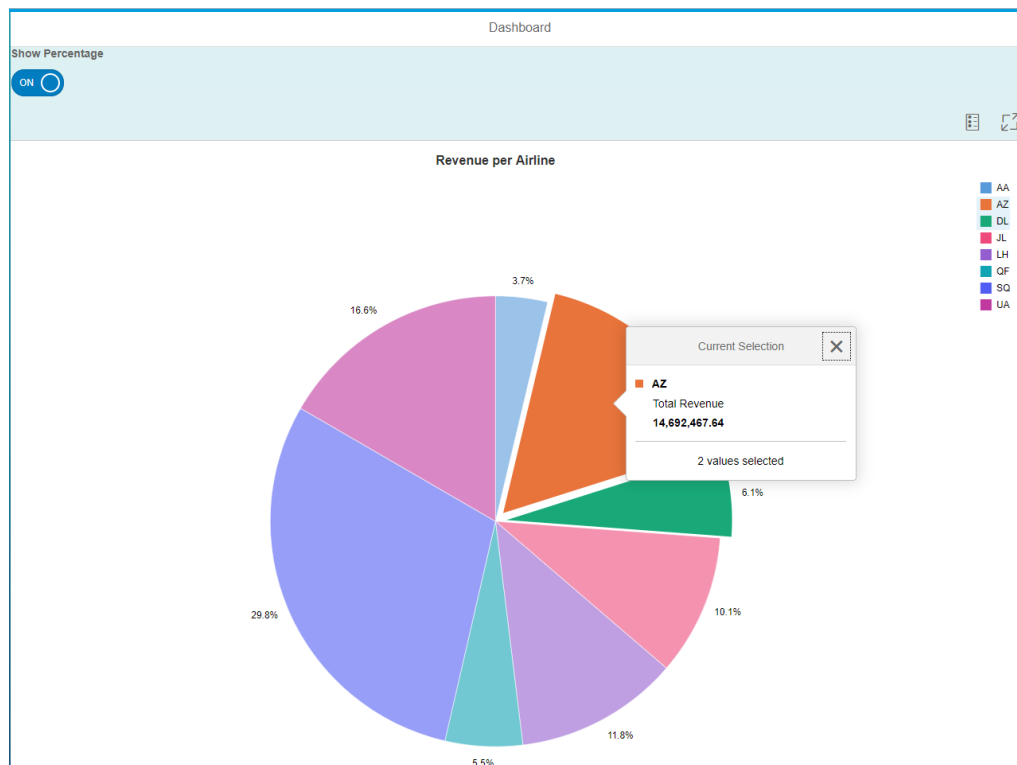
To test your application, first make sure that all source files were correctly submitted. For this, right-click on your project, and click on **Team->Submit**. Choose all your files and click on **Finish**. If **submit** is already greyed out, everything is fine.

Finally, right-click on your project, and choose **Run As->Run on ABAP Server**.

You may need to login with your username and password. Afterwards, your application will be displayed in the integrated web browser. There, you will see your pie chart. If you want to view your application in the system browser (like for debugging purposes), you

**Test your
application**

can click on .



Loading all the necessary data might take some time. When you see the chart, experiment with the percentage switch, toolbar buttons, and with clicking on the chart elements.

Task 5: Develop a service for a smart chart

Short description: Develop another OData service and annotate it to make it consumable by the smart UI controls, such as smart chart.

In the integrated SAP GUI call transaction code **SEGW** again and create a new project, called **Z_000_ANALYTICS**. Be sure to choose the project type “Service with Vocabulary-Based Annotations”.

Create Project

*Project: Z_000_ANALYTICS

*Description: Analytical Project for EX14

Attributes

*Project Type: 2 Service with Vocabulary-Based Annotations

Generation Strategy: 1 Standard

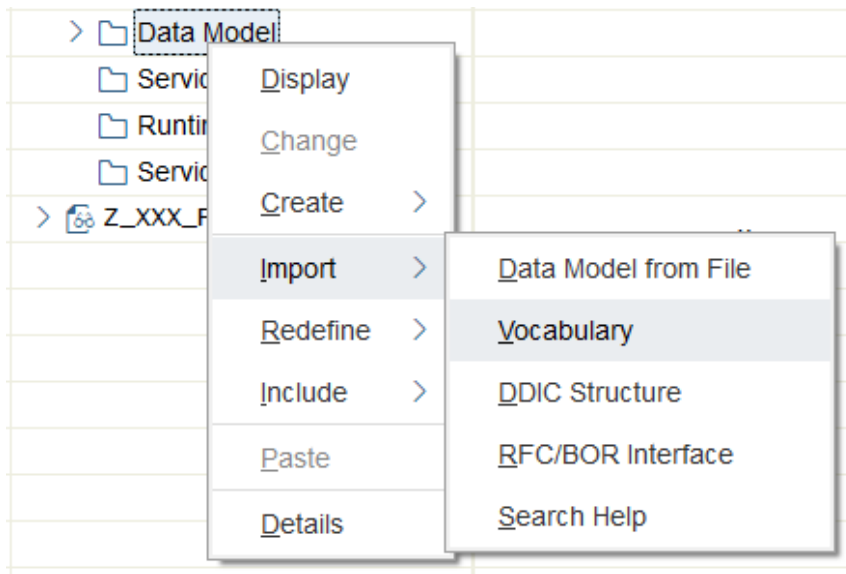
Object Directory Entry

*Package: Z_ABAP

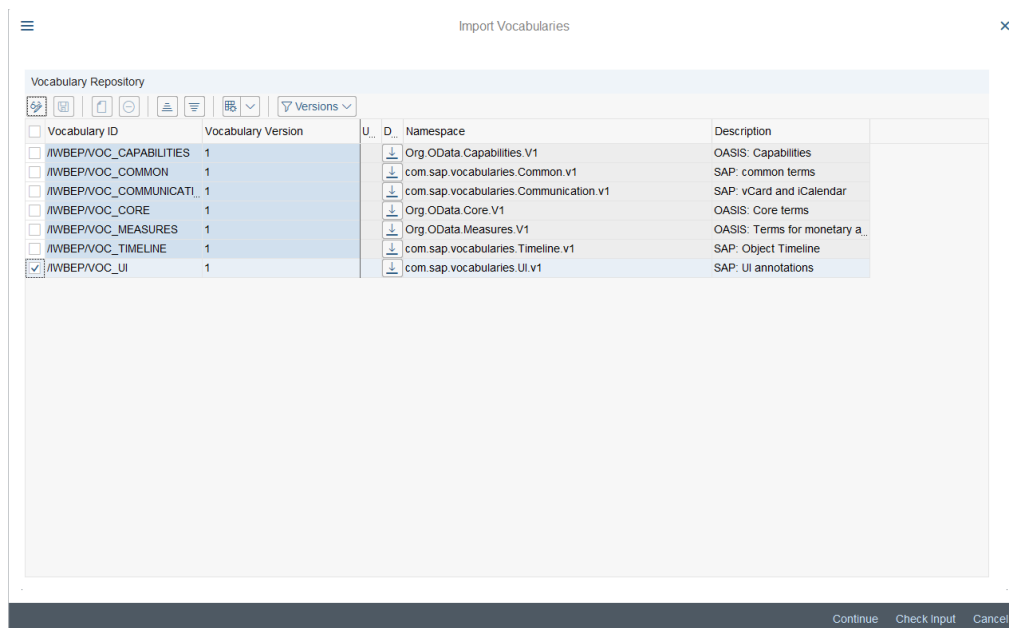
*Person Responsible: BD0D1

Continue Local Object Cancel

Expand the newly created project, right-click on Data Model, and import a Vocabulary.



In the next window choose the last entry, **/IWBEP/VOC_UI**, and click Continue.



Now, create a new entity like in the task 1. Call it **Savgprice** and base it on the ABAP structure **SBOOK**. Select only **Carrid** and **Connid** from the list of the fields and mark them both as keys. In this chart we will display the average price per airline and connection, and we will add the necessary field manually. Expand the Entity Types node and the newly created type. Double-click on properties and you will see both Carrid and Connid in the table on the right. Create a new property by clicking on Append Row . Call it **Loccuram** of type **Edm.Decimal** with precision **15** and scale **2** as well as **LOCCURAM** as ABAP Field name.

Properties										
Name	Is Key	Type Kind	Data Type	Prec.	Scale	Max	Fix L	Nulla	ABAP Field Name	A. Colle...
Carrid	☒	Core Type	Edm.String	0	0	3	☐	☐	CARRID	☐
Connid	☒	Core Type	Edm.String	0	0	4	☐	☐	CONNID	☐
Loccuram	☐	Core Type	Edm.Decimal	15	2	0	☐	☐	LOCCURAM	☐

Save your project and generate runtime artifacts.

We will need the query method to fetch the data from the database like we did in the previous tasks. Again, right-click on the **...DPC_EXT** class, and choose **“Go to ABAP Workbench”**. Define the query method in the protected section:

PROTECTED SECTION.

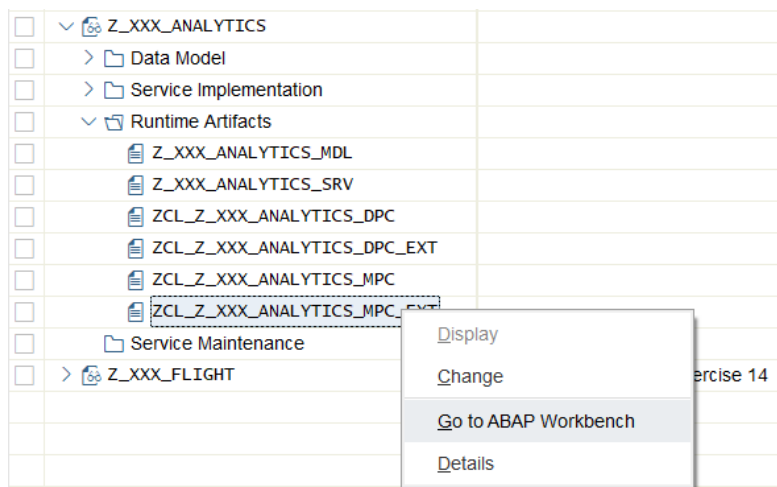
METHODS savgpriceset_get_entityset **REDEFINITION.**

Implement the method. Here you have to select the average price from the table SBOOK, based on the two key fields, Carrid, and Connid, and group the results by both key fields.

```
METHOD savgpriceset_get_entityset.  
  SELECT carrid, connid, AVG( loccuram ) AS loccuram  
  INTO CORRESPONDING FIELDS OF TABLE @et_entityset  
  FROM sbook  
  GROUP BY carrid, connid  
  ORDER BY carrid, connid.  
ENDMETHOD.
```

Save and activate your class.

This implementation is sufficient for the simple chart. But since we are planning to use a smart chart, we need to define our metadata. It is possible in the SAP gateway, but this method is time-consuming and not very reliable. That is why we are implementing the necessary annotations in the **...MPC_EXT** class. Open it in ABAP workbench.



We are going to redefine two methods: **DEFINE** and **DEFINE_VOCAB_ANNOTATIONS**. The first method is public, and the second is protected.

```
public section.  
  METHODS define REDEFINITION.  
protected section.  
  METHODS define_vocab_annotations REDEFINITION.
```

First, we are going to define the semantics for our entity type – it is an **aggregation**. Its fields need an aggregation role. Carrid and Connid will act as **dimensions**, and Loccuram is a **measure**. Implement the DEFINE method:

METHOD define.

```
DATA: lo_entity_type TYPE REF TO /iwbsp/if_mgw_odata_entity_typ,
      lo_property     TYPE REF TO /iwbsp/if_mgw_odata_property,
      lo_anno         TYPE REF TO /iwbsp/if_mgw_odata_annotation.

CALL METHOD super->define( ).

lo_entity_type = model->get_entity_type( iv_entity_name = 'Savgprice' ).

CALL METHOD lo_entity_type->/iwbsp/if_mgw_odata_annotatabl~create_annotation(
  EXPORTING iv_annotation_namespace = /iwbsp/if_mgw_med_odata_types=>gc_sap_namespace
  RECEIVING ro_annotation           = lo_anno ).
lo_anno->add( iv_key = 'semantics' iv_value = 'aggregate' ).

lo_property ?= lo_entity_type->get_property( iv_property_name = 'Carrid' ).
CALL METHOD lo_property->/iwbsp/if_mgw_odata_annotatabl~create_annotation(
  EXPORTING iv_annotation_namespace = /iwbsp/if_mgw_med_odata_types=>gc_sap_namespace
  RECEIVING ro_annotation           = lo_anno ).
lo_anno->add( iv_key = 'aggregation-role' iv_value = 'dimension' ).

lo_property ?= lo_entity_type->get_property( iv_property_name = 'Connid' ).
CALL METHOD lo_property->/iwbsp/if_mgw_odata_annotatabl~create_annotation(
  EXPORTING iv_annotation_namespace = /iwbsp/if_mgw_med_odata_types=>gc_sap_namespace
  RECEIVING ro_annotation           = lo_anno ).
lo_anno->add( iv_key = 'aggregation-role' iv_value = 'dimension' ).

lo_property ?= lo_entity_type->get_property( iv_property_name = 'Loccuram' ).
CALL METHOD lo_property->/iwbsp/if_mgw_odata_annotatabl~create_annotation(
  EXPORTING iv_annotation_namespace = /iwbsp/if_mgw_med_odata_types=>gc_sap_namespace
  RECEIVING ro_annotation           = lo_anno ).
lo_anno->add( iv_key = 'aggregation-role' iv_value = 'measure' ).

ENDMETHOD.
```

Now we need to create the annotation for the chart. Define the chart title, description, and type, as well as the lists of measures and dimensions. Pay attention to your number when setting the namespace qualifier.

METHOD define_vocab_annotations.

```
DATA: lo_ann_target TYPE REF TO /iwbsp/if_mgw_vocan_ann_target.
DATA: lo_annotation TYPE REF TO /iwbsp/if_mgw_vocan_annotation.
DATA: lo_collection TYPE REF TO /iwbsp/if_mgw_vocan_collection.
DATA: lo_function   TYPE REF TO /iwbsp/if_mgw_vocan_function.
DATA: lo_fun_param  TYPE REF TO /iwbsp/if_mgw_vocan_fun_param.
DATA: lo_property   TYPE REF TO /iwbsp/if_mgw_vocan_property.
DATA: lo_record     TYPE REF TO /iwbsp/if_mgw_vocan_record.
DATA: lo_record1    TYPE REF TO /iwbsp/if_mgw_vocan_record.
DATA: lo_simp_value TYPE REF TO /iwbsp/if_mgw_vocan_simple_val.
DATA: lo_url        TYPE REF TO /iwbsp/if_mgw_vocan_url.
DATA: lo_label_elem TYPE REF TO /iwbsp/if_mgw_vocan_label_elem.
DATA: lo_reference  TYPE REF TO /iwbsp/if_mgw_vocan_reference.

lo_ann_target = vocab_anno_model->create_annotations_target( 'Savgprice' ).
"replace XXX with your number
lo_ann_target->set_namespace_qualifier( 'Z_XXX_ANNLYTICS_SRV' ).
lo_annotation = lo_ann_target->create_annotation( iv_term = 'com.sap.vocabularies.UI.v1.Chart' ).
lo_record = lo_annotation->create_record( iv_record_type = 'com.sap.vocabularies.UI.v1.ChartDefinitionType' ).
lo_record1 = lo_record.

lo_property = lo_record->create_property( 'Title' ).
lo_simp_value = lo_property->create_simple_value( ).
lo_simp_value->set_string( 'Price Chart' ).

lo_property = lo_record->create_property( 'Description' ).
lo_simp_value = lo_property->create_simple_value( ).
lo_simp_value->set_string( 'Average Price Per Airline and Connection' ).

lo_property = lo_record->create_property( 'ChartType' ).
lo_simp_value = lo_property->create_simple_value( ).
lo_simp_value->set_enum_member_by_name( 'com.sap.vocabularies.UI.v1.ChartType/Column' ).

lo_property = lo_record->create_property( 'Measures' ).
lo_simp_value = lo_property->create_simple_value( ).
lo_collection = lo_property->create_collection( ).
lo_simp_value = lo_collection->create_simple_value( ).
lo_simp_value->set_property_path( 'Loccuram' ).

lo_property = lo_record->create_property( 'Dimensions' ).
lo_simp_value = lo_property->create_simple_value( ).
lo_collection = lo_property->create_collection( ).
lo_simp_value = lo_collection->create_simple_value( ).
lo_simp_value->set_property_path( 'Carrid' ).

lo_simp_value = lo_collection->create_simple_value( ).
lo_simp_value->set_property_path( 'Connid' ).
```


Continue with the method implementation – define the measure attributes and dimension attributes, such as the roles and the paths to each attribute inside of the entity. Both of our dimensions are **categories**, and the measure should belong to **Axis1**.

```
lo_record = lo_collection->create_record( iv_record_type = 'com.sap.vocabularies.UI.v1.ChartDimensionAttributeType' ).
lo_property = lo_record->create_property( 'Dimension' ).
lo_simp_value = lo_property->create_simple_value( ).
lo_simp_value->set_property_path( 'Carrid' ).
lo_property = lo_record->create_property( 'Role' ).
lo_simp_value = lo_property->create_simple_value( ).
lo_simp_value->set_enum_member_by_name( 'Category' ).

lo_record = lo_collection->create_record( iv_record_type = 'com.sap.vocabularies.UI.v1.ChartDimensionAttributeType' ).
lo_property = lo_record->create_property( 'Dimension' ).
lo_simp_value = lo_property->create_simple_value( ).
lo_simp_value->set_property_path( 'Connid' ).
lo_property = lo_record->create_property( 'Role' ).
lo_simp_value = lo_property->create_simple_value( ).
lo_simp_value->set_enum_member_by_name( 'Category' ).

lo_property = lo_record1->create_property( 'MeasureAttributes' ).
lo_simp_value = lo_property->create_simple_value( ).
lo_collection = lo_property->create_collection( ).
lo_record = lo_collection->create_record( iv_record_type = 'com.sap.vocabularies.UI.v1.ChartMeasureAttributeType' ).
lo_property = lo_record->create_property( 'Measure' ).
lo_simp_value = lo_property->create_simple_value( ).
lo_simp_value->set_property_path( 'Loccuram' ).
lo_property = lo_record->create_property( 'Role' ).
lo_simp_value = lo_property->create_simple_value( ).
lo_simp_value->set_enum_member_by_name( 'Axis1' ).
```

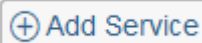
ENDMETHOD.

Save and activate your class.

Task 6: Activate the service and check the metadata

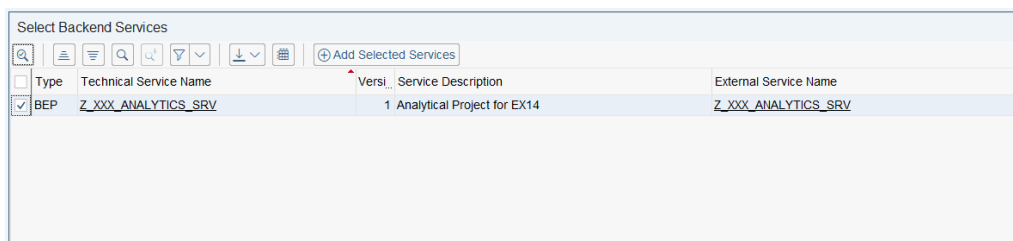
Short description: We have developed an OData service and annotated it so that the smart chart can understand the data, fetched via that service. To be consumable, the service needs to be activated. It is also necessary to check whether the annotations have been understood correctly and whether they are depicted correctly in the service metadata.

To activate the service, open the **/n/IWFND/MAINT_SERVICE** transaction. There click

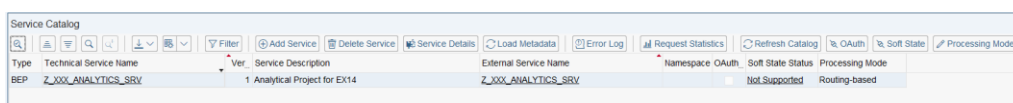


on Add service . In the opened window enter LOCAL as System Alias and Z_XXX_ANALYTICS_SRV as Technical Service Name (replace XXX with your number), and press Enter. Your service should be listed in the selected services section.

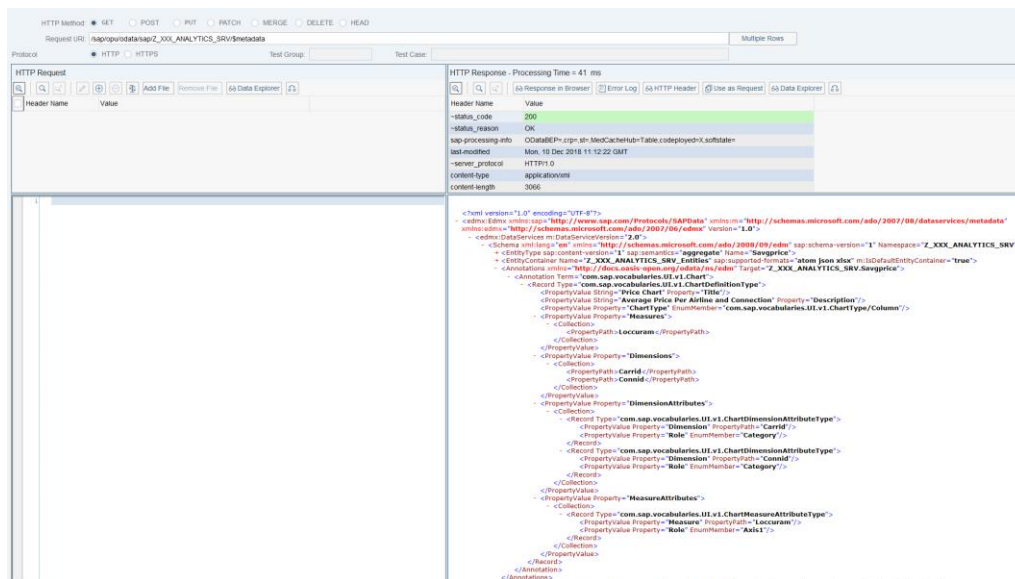
Click on the line with it and click on Add Selected Services



In the next window leave the names without changes and select your package. When prompted for the transport request, make sure that your request is selected. After receiving a success message click Continue. Click on Back  to return to the service catalog. Find your service in the list (or alternatively filter for it), and select the line.



In the bottom part of the screen click on SAP Gateway Client . There we have to check the metadata of the service. Add the **\$metadata** to the URI and click on Execute.

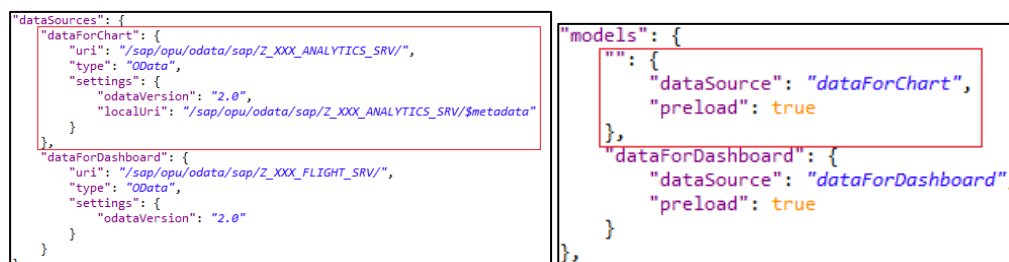


You can see the correspondence between your implementation and the results of your call. The annotation for the chart is located under the entity type and the entity container definitions.

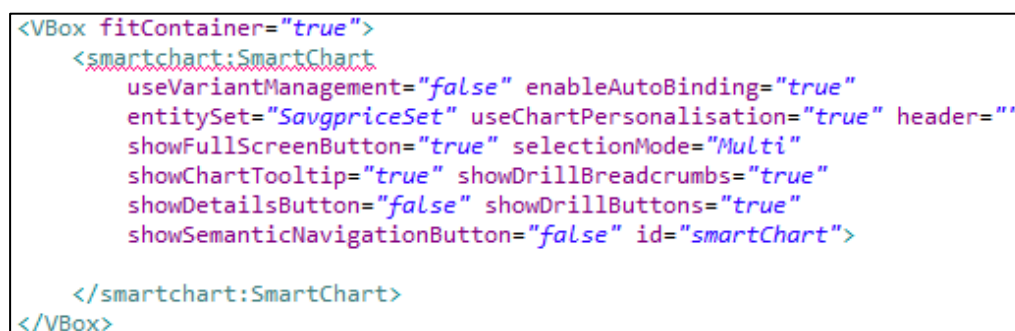
Task 7: Extend the UI5 application with the smart chart

Short description: We have a running UI5 application, depicting the revenue per airline in a form of a pie chart. We have also developed and activated an OData service that can provision the data for a smart chart. Now the application has to be extended with this UI control.

First step is to declare a new data source in the **manifest.json** file and to reference the data source in the default data model. Add the **dataForChart** data source and default ("" model. Be sure to use your own service.



Add the control definition to a view after the FixFlex:



Resolve the error by adding a missing dependency to the view header:

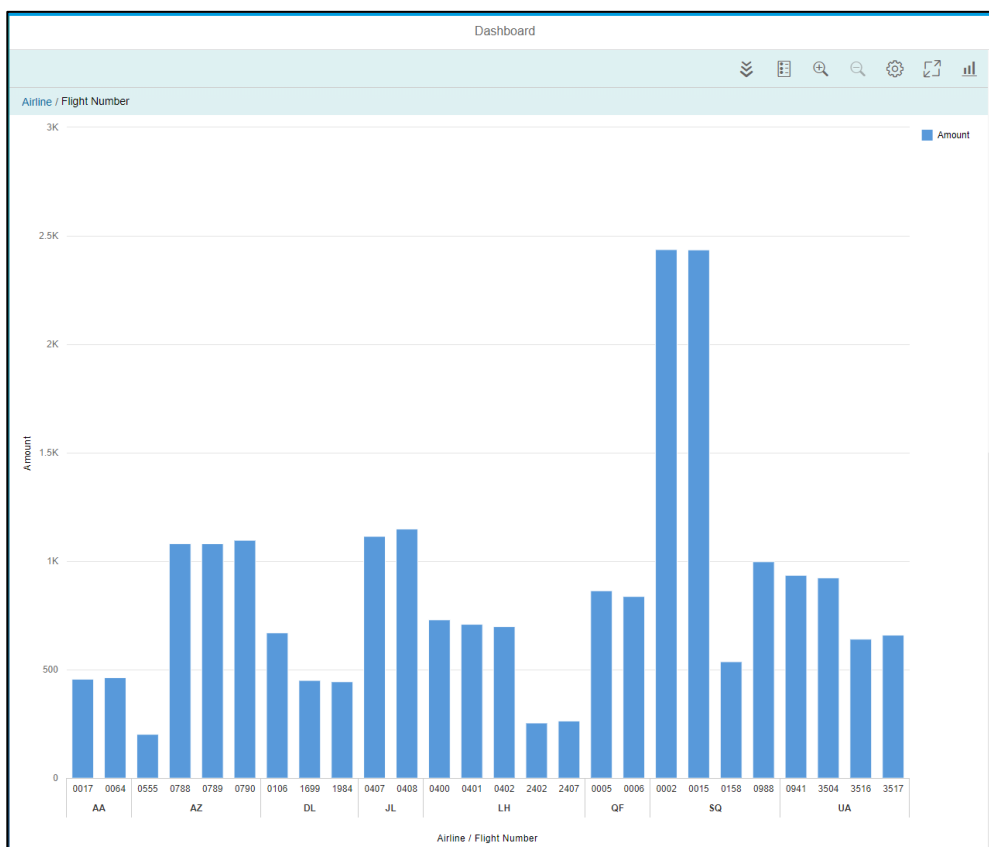
```
xmlns:smartchart="sap.ui.comp.smartchart"
```

The last step is to fetch the data from the database. In the **onInit** function of the controller, perform the query method against the defined set, and start the mock server to serve the data to the smart chart if the result has been successful:

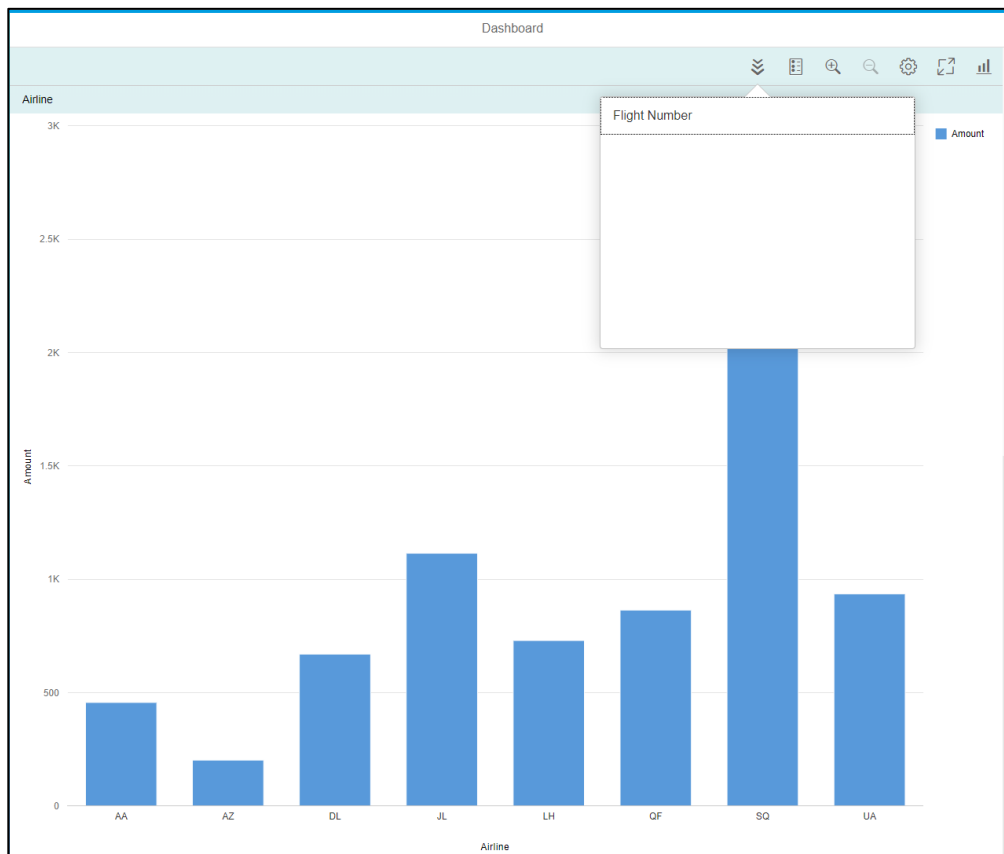
```
var serviceURL = that.getOwnerComponent().getModel().getServiceUrl+"/";
var oOdataModel = new sap.ui.model.odata.v2.ODataModel(serviceURL);
oOdataModel.read("/SavgpriceSet", {
    success: function(oData, oResponse){
        console.log(oData.results);
        var oMockServer = new MockServer({
            rootUri: serviceURL
        });
        that._oMockServer = oMockServer;
        oMockServer.simulate(serviceURL+"$metadata");
        oMockServer.setEntitySetData("SavgpriceSet", oData.results);
        oMockServer.start();
    }
});
```

Unlike the OData exercise, where we've defined a service URL in a separate file, here we load it directly from the controller metadata. This value gets populated from the manifest. After querying the database, we use the API of the mock server to add our data to it and to start it.

Save all your files and submit your changes to the backend. After that start the application. For the best experience, open it in browser. You should see the smart chart underneath the pie chart.



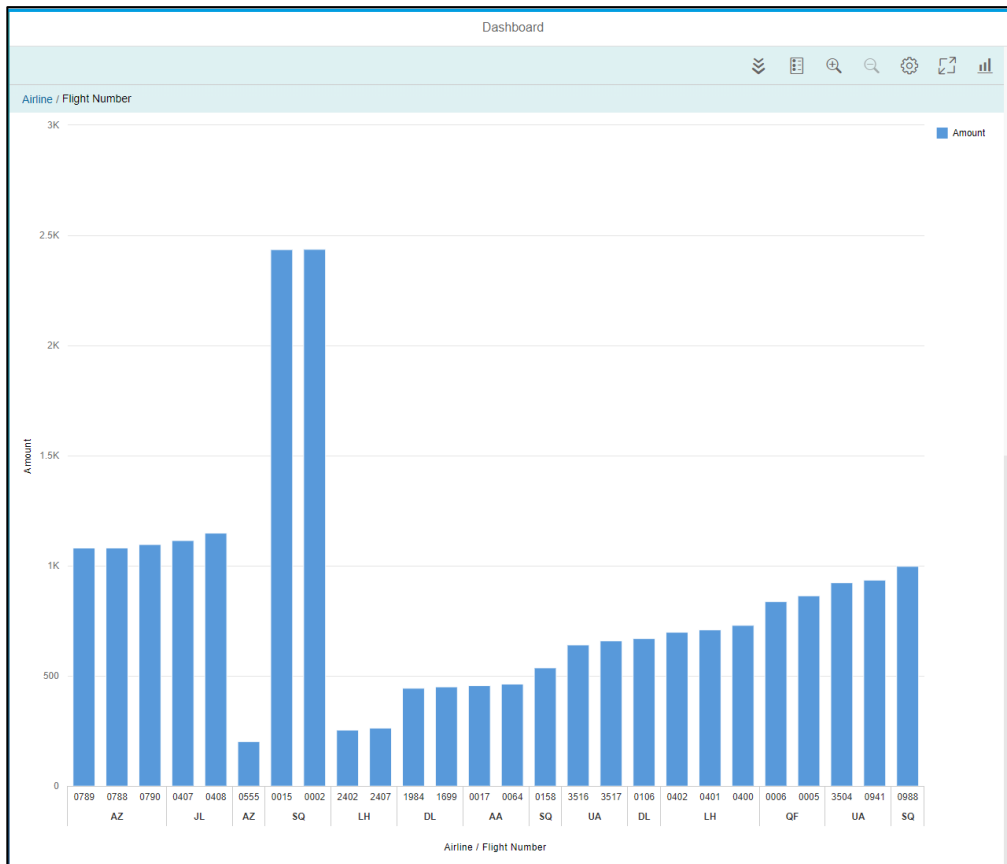
Experiment with the roll-up and drill-down buttons:



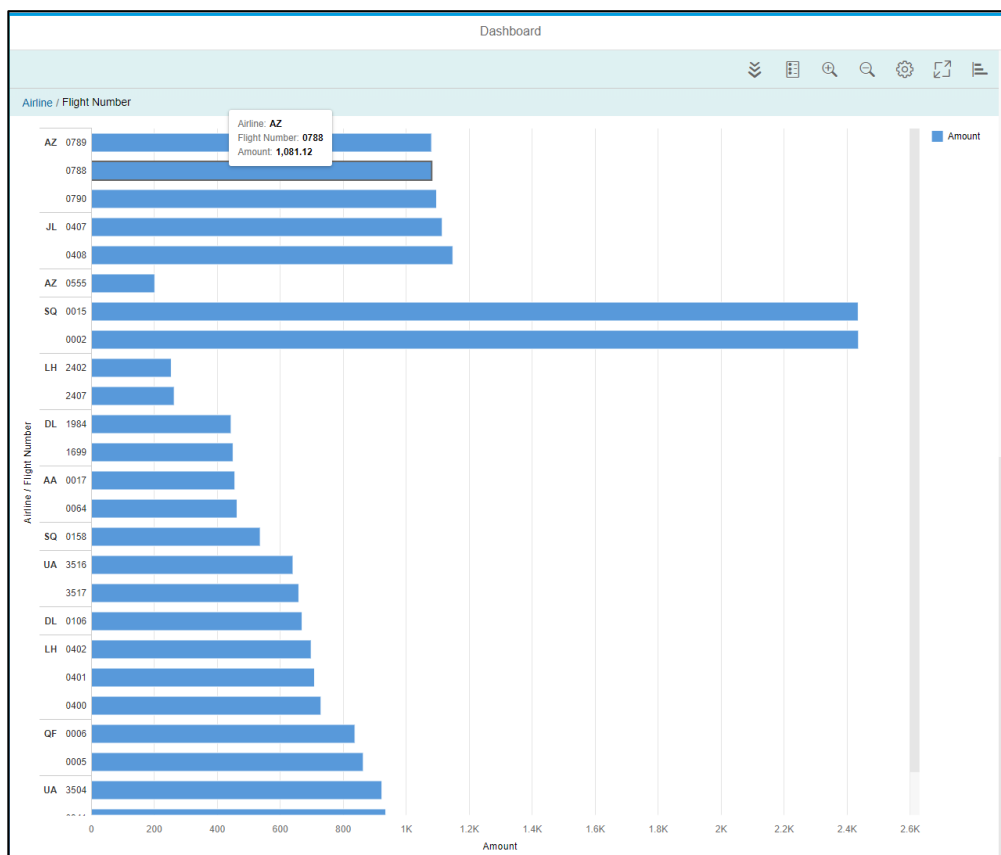
Try out the sorting and filtering options (press on Settings ):

View Settings		
Chart	Sort	Filter
Amount	Ascending	
Flight Number	Descending	

OK Cancel Restore



You can also change the chart type by pressing on . Convert it to a bar chart:



Challenge: Modularize your application

Currently we have developed both charts in one view, strictly following the blob development anti-pattern (<http://antipatterns.com/briefing/sld024.htm>). This is a bad practice, that is why you have to modularize the application. Create a structure, similar to the SAP UI5 exercise – a main view with two buttons, “Show Pie Chart” and “Show Smart Chart”. Move the charts from the main view into an own view. Implement navigation using hashing, and enable navigation to the respective view when clicking on the button.